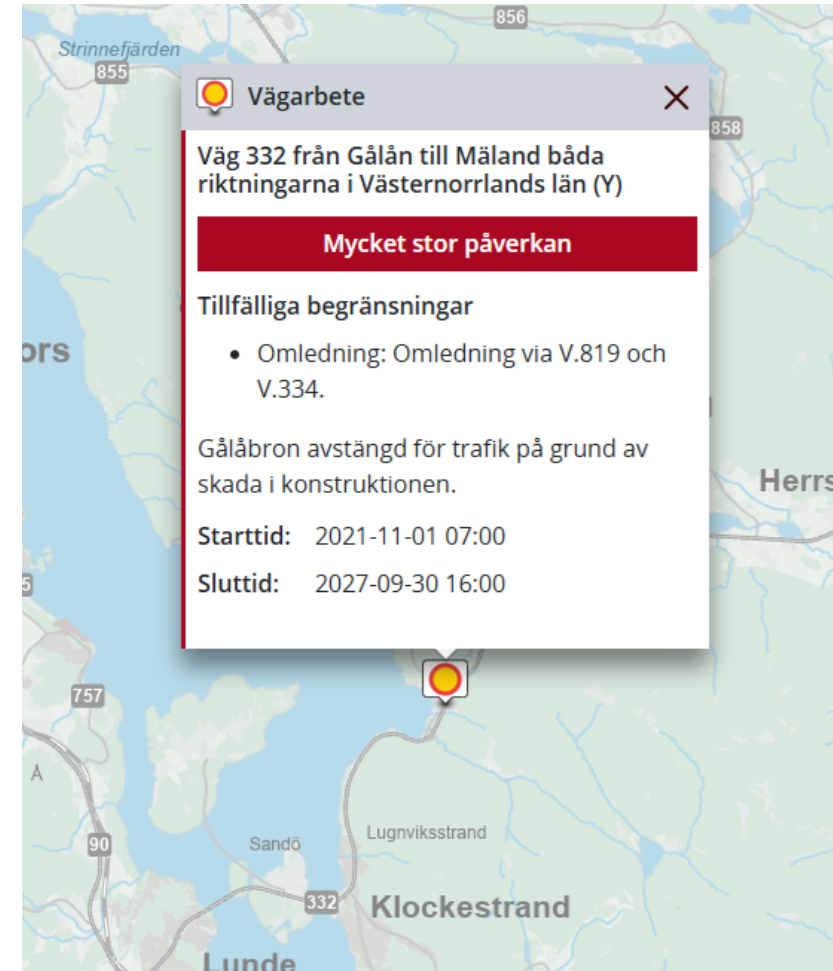


Realtidsdata i Origo

Automatisk uppdatering av kartlager i Origo när underliggande data ändras, exempelvis trafikverkets trafiksituation

Användningsfallet

- Hösten 2025 drabbades Örnsköldsvik av kraftiga regn som spolade bort vägar och gjorde att hemtjänsten hade svårt att nå sina brukare
- Trafikverket har en karta med aktuell trafikinformation, men i den syns inte brukarnas adress
- Verksamheten behövde en kartapplikation som både visar var brukarna finns och vilka vägar som är avstängda

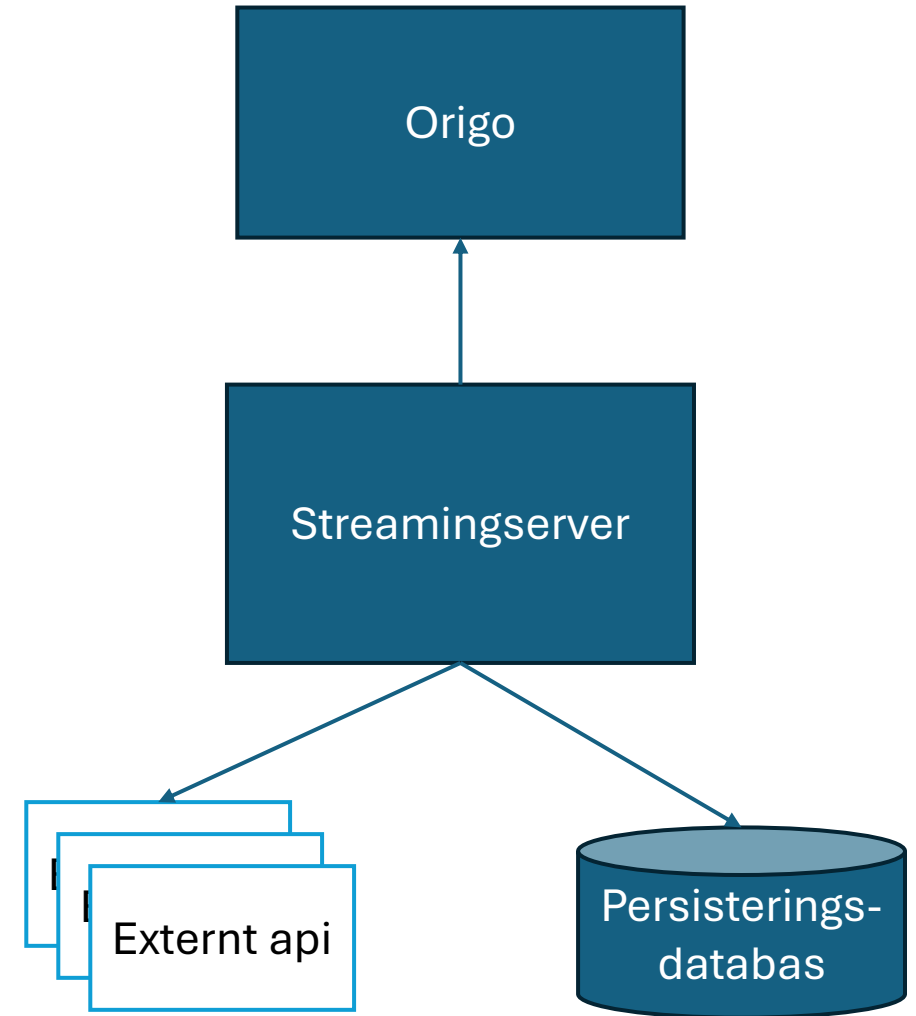


Problemet

- Origo uppdaterar inte automatiskt ett kartlager om underliggande data uppdateras
- En periodisk refresh belastar både server och surfpott i onödan
- Manuell refresh är bökigt
- Existerande karttjänster (ex. wms/wfs) saknar mekanismer för att skicka events
- Det finns ingen etablerad standard eller serverprodukt som tillhandahåller realtidskartlager
- Det finns en uppsjö av olika realtidsdatakällor med olika protokoll och egenskaper som gör det omöjligt att göra en lagertyp i Origo som passar för alla realtidsdatakällor om man vill göra detta generellt
 - Vissa typer av källor går inte ens att ansluta till från webbläsare

Lösningen

- Ett generellt realtidslager i Origo som kan lyssna på events och uppdatera kartan när det kommer event
- Specificerat ett protokoll för att kunna bygga en enhetlig server side oavsett datakälla
 - Bygger på öppna standarder
- Man måste alltså implementera sin egen streamingserver som en proxy mot realtidskällan
 - För vanliga källor skulle man kunna samarbeta t.ex i Origo-server eller ett fristående projekt.
 - Förhoppningsvis kommer det att komma standardprodukter som använder samma öppna standarder



Tekniken

- Lagret ansluter till en *OGC Features API* (wfs 3.0) endpoint som har möjlighet att skicka adressen till en ström
- Events skickas via *Websockets*. Även *SSE* skulle kunna användas, men vi implementerade bara websockets
- Eventen är av typen *CloudEvents* med *GeoJson* payload
 - Create/replace/delete

Features API response

```
{
  "type": "FeatureCollection",
  "features": [],
  "links": [
    {
      "href": "wss://karta-utv.kommun.ornskoldsvik.se/origorealttime/streams?layer=demoavbrott",
      "rel": "hub"
    }
  ]
}
```

Exempel på CloudEvent

```
{
  "id": 2,
  "source": "demosituation",
  "specversion": "1.0",
  "type": "org.ogc.api.collection.item.replace",
  "datacontenttype": "application/json",
  "data": {
    "type": "Feature",
    "geometry": {
      "type": "Point",
      "coordinates": [
        18.436483,
        63.24287124
      ]
    },
    "properties": {
      "fid": 27,
      "fritext": "Bron är bortspolad"
    }
  },
  "id": "punktlager.27"
}
```

Server side implementationen

- Exponerar en Features API GET collection endpoint (ett "lager") per datakälla
 - Innehåller en websocket URL i svaret
 - Man behöver inte implementera hela Features API specifikationen
 - Det fiffiga med att inte ansluta mot websocket direkt är att Features API-svaret kan byta URL för att hantera sessionsparametrar etc
- Hämtar data från realtidskälla med valfri teknik beroende på vad källan stödjer
 - MQTT, HTTP GET polling, SSE, websockets, SQL, Rabbit MQ, whatever
 - Kan antingen göras i streamingservern eller som en extern process, t.ex. FME som trycker in i en databas.
- Skickar events till klienterna när data ändrats
- Beroende på datats egenskaper implementerar man server siden olika
 - Data som uppdateras frekvent (t.ex bussars position) kan man skicka senaste utan att ta hänsyn till omsändningar eller initialt tillstånd vid anslutning
 - Data som uppdateras sällan (t.ex. trafiksituation) kan man behöva spara i en databas för att kunna skicka som initialt tillstånd när nya klienter ansluter och återanslutningar
 - Data som har stora mängder kan man behöva hantera omsändningar för att slippa skicka allt data som initialt tillstånd vid återanslutningar efter tappad anslutning
- Med en egen server side har man full kontroll över
 - Autentisering av klienter
 - Auktorisering av användare
 - API-nycklar mot datakällor (som kuriosa har TRV sina egna API-nycklar fullt läsbara client side)
 - CORS

Status Örnsköldsvik

- Implementerat en lagertyp i Origo
 - För närvarande i PR som ej mergats in
- Implementerat en server side i nodejs
 - Modulärt uppbyggd med ett ramverk som stödjer enkel implementation av ytterligare datakällor.
 - Ville helst göra server side med FME, men hade inte råd att hogga en enginelicens 24/7
- Hittills bara trafiksituation i produktion
 - Fast TRV:s api stödjer events (via SSE) pollar vi och lagrar i en databas och eventar periodiskt
 - Server side döljer ändå detta mot klienten som uppfattar det som eventdrivet
- Planer på att hämta data från kommunens IoT-plattform (IoTopen)
 - Skulle öppna upp för en mängd givare
- Undersöker möjlighet att få ut pågående strömavbrott.
 - I dagsläget har ingen av de nätägare som agerar i kommunen API för det.
- Idé om att hämta lokaltrafikens position från Trafiklab
- Kikar på SMHI:s apier

Demo

- Mockad server side eftersom det är lite svårt att veta när det händer olyckor
- Förinspelat för att snabba på presentationen, men det är på riktigt

